

CENTRO UNIVERSITÁRIO IESB
SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS

LAÍS SALES XAVIER

ALGORITMOS E PROGRAMAÇÃO DE COMPUTADORES
ATIVIDADE ATIVA

BRASÍLIA, DF

2026

SUMÁRIO

1. DISPONIBILIZAÇÃO DO PROJETO.....	3
2. ORGANIZAÇÃO DOS ARQUIVOS.....	3
3. DEPENDÊNCIAS.....	3
4. USO DO SISTEMA.....	4
5. FUNCIONALIDADES.....	4
5.1. Menu Principal (menu.py).....	5
5.2. Cardápio (cardapio.py).....	6
5.3. Cadastro (cadastro.py).....	8
5.4. Pedido (pedido.py).....	9
6. CONCLUSÃO.....	13

1. DISPONIBILIZAÇÃO DO PROJETO

O projeto está disponibilizado no GitHub, com acesso público, no link:
<https://github.com/laiszz/IESB-Algoritmos-AA>

2. ORGANIZAÇÃO DOS ARQUIVOS

O projeto está disposto em arquivos organizados por funcionalidades.

1. **ARQUIVO PRINCIPAL:** O arquivo principal *main.py*, usado para executar o programa, está localizado no diretório **raiz**;
2. **ARQUIVOS DE OBJETOS:** Os arquivos de objetos, onde são definidas as Classes usadas no projeto e seus respectivos atributos e métodos, estão no diretório “**objetos**”;
3. **ARQUIVOS AUXILIARES:** Os arquivos auxiliares, que contém as funções necessárias para carregar as telas (prints) e lógicas durante a execução do programa, estão no diretório “**auxiliares**”;
4. **ARQUIVOS DE DOCUMENTAÇÃO:** Os arquivos de documentação, como o fluxograma detalhado dos acionamentos de funções, esse próprio documento e um vídeo demonstrativo (zipado), estão no diretório “**documentacao**”. Além disso, existe um *README.md* localizado no diretório **raiz**.

3. DEPENDÊNCIAS

Para garantir o funcionamento correto do programa, são necessários os pacotes externos “*readchar*” e “*tabulate*”:

1. **Readchar:** esse pacote é uma alternativa ao método nativo do Python “*input*”, de modo que o usuário não precisa pressionar a tecla Enter para enviar os dados digitados (é lida apenas a primeira tecla digitada). Para baixá-lo, execute o comando ``pip install readchar`` no diretório raiz do projeto.
2. **Tabulate:** é um pacote para formatação de tabelas, facilitando a impressão (prints) de dados que precisam de uma organização extra — como é o caso do resumo do pedido. Para baixá-lo, execute o comando ``pip install tabulate`` no diretório raiz do projeto.

4. USO DO SISTEMA

Para iniciar o programa, garanta que as dependências foram instaladas corretamente, abra um terminal (CMD no Windows, por exemplo) e execute o comando ``python3 main.py`` no diretório raiz do projeto. O projeto foi desenvolvido na versão 3.14.6 do Python, sendo necessário garantir uma execução do programa com o *python3*.

Para navegar entre as opções apresentadas, utilize os números do teclado ou o NumPad. Por exemplo: no *Menu Principal*, basta apertar o número **1** no teclado/NumPad para prosseguir com a visualização do cardápio. Não é necessário apertar Enter graças a implementação do pacote externo “*readchar*”. Qualquer outra tecla pressionada que não seja um dos números das opções (como apertar uma letra, SHIFT, ESC, número maior ou menor que as opções disponíveis, etc.) trará a mesma tela em loop, dando a sensação de que aqueles botões “não funcionam” e o sistema só responde aos botões corretos.

Caso deseje encerrar o programa, é possível pressionar ``CTRL+C`` a qualquer momento ou o número **0** no *Menu Principal*.

5. FUNCIONALIDADES

A ideia por trás do projeto é disponibilizar à empresa fictícia Tia Rosa Coffee Shop um programa capaz de: mostrar o cardápio, cadastrar clientes, e registrar pedidos de clientes. O programa contém, então, um *Menu Principal* que fornece essas 3 opções ao usuário.

Para facilitar a usabilidade do sistema por pessoas menos experientes, são usadas telas fixas em loop — implementando *funções recursivas* ao invés de *laços de repetição* — e o pacote externo “*readchar*” na leitura de inputs de opção — para que seja possível navegar pressionando apenas o número da opção desejada. Todos os fluxos (cardápio, cadastro e pedido), ao seu final, retornam para o *Menu Principal* para que o usuário tenha uma experiência contínua no sistema sem a necessidade de reiniciar o programa.

Um fluxograma completo com todos os acionamentos e caminhos possíveis do usuário está presente no diretório de documentação do projeto.

5.1. Menu Principal (menu.py)

O Menu Principal consiste em uma arte com caracteres ASCII (gerada no site <https://patorjk.com/software/taag/>) e opção pelas 3 funcionalidades primárias do programa — além da opção de Sair, que encerra o programa.

```
Prompt de Comando - python  +  v
*****
The Coffee Shop
*****

Aperte um dos números abaixo para selecionar uma opção:
1 - Ver o cardápio
2 - Fazer um pedido
3 - Cadastrar cliente
0 - Sair
```

Imagem 1. Print do Menu Principal com a arte em ASCII e as opções.

5.2. Cardápio (cardapio.py)

O Cardápio possui duas apresentações: um cardápio completo, e um cardápio promocional. Essas apresentações são escolhidas pelo usuário logo na primeira tela, redirecionando-o para a opção desejada.

```
Prompt de Comando - pythor x + v
Deseja ver o cardápio completo ou apenas as promoções?

Aperte um dos números abaixo para selecionar uma opção:
1 - Ver o cardápio completo
2 - Ver o cardápio das promoções
|
```

Imagem 2. Print das opções de cardápio disponíveis.

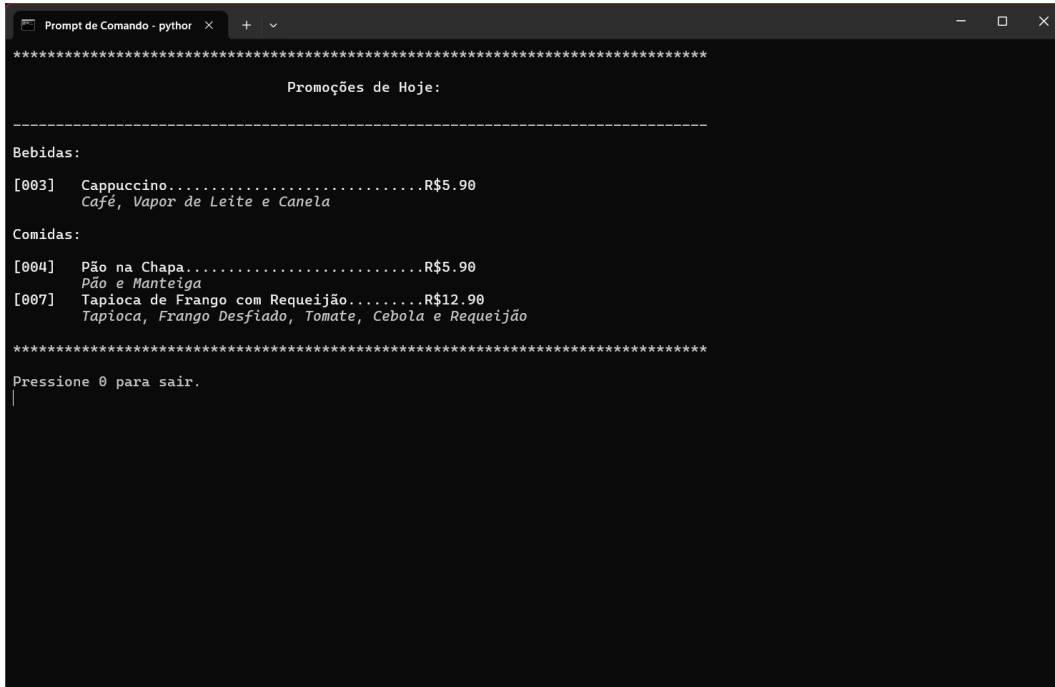
Os produtos são dispostos em categorias (“Bebidas” e “Comidas”), com um layout que mostra o código, nome, ingredientes e preço de cada um. Isso facilita o trabalho do funcionário que registrará o produto no pedido (pelo código) e disponibiliza informações dos produtos para o cliente.

O cardápio padrão contém todos os produtos e destaca aqueles que estão em promoção (asteriscos após o preço).

```
Prompt de Comando - pythor x + v
*****
Cardápio:
-----
Obs: Produtos destacados com *** estão na promoção!
Bebidas:
[001]  Café Expresso.....R$2.90
       Café
[002]  Pingado.....R$3.30
       Café e Leite
[003]  Cappuccino.....R$5.90***
       Café, Vapor de Leite e Canela
Comidas:
[004]  Pão na Chapa.....R$5.90***
       Pão e Manteiga
[005]  Pão com Queijo Quente.....R$12.90
       Pão, Manteiga e Queijo Minas Frescal
[006]  Misto Quente.....R$9.90
       Pão, Manteiga, Presunto e Queijo Mussarela
[007]  Tapioca de Frango com Requeijão.....R$12.90***
       Tapioca, Frango Desfiado, Tomate, Cebola e Requeijão
[008]  Fatia de Bolo de Chocolate.....R$7.90
       Cacaú 50%, Açúcar, Ovo, Farinha de Trigo, Óleo, Leite e Fermento Químico
*****
Pressione 0 para sair.
|
```

Imagem 3. Print do cardápio completo.

O cardápio promocional apresenta apenas os produtos em promoção.



```
Prompt de Comando - pythor x + v
*****
                          Promoções de Hoje:
-----
Bebidas:
[003]  Cappuccino.....R$5.90
       Café, Vapor de Leite e Canela

Comidas:
[004]  Pão na Chapa.....R$5.90
       Pão e Manteiga
[007]  Tapioca de Frango com Requeijão.....R$12.90
       Tapioca, Frango Desfiado, Tomate, Cebola e Requeijão

*****
Pressione 0 para sair.
|
```

Imagem 4. Print do cardápio promocional. Aqui os itens não são mais destacados com asteriscos e o título do cardápio muda para “Promoções de Hoje”.

5.3. Cadastro (cadastro.py)

A opção de cadastrar clientes é simples, sendo necessário apenas informar o CPF e Nome do cliente. Os campos para digitação usam o método nativo do Python “input” para coletar as informações como string.

```
Prompt de Comando - pythor x + v
*****
                          Cadastro de Cliente:
                          -----

Preencha com as informações do Cliente:
Nome do Cliente:
Lais Sales Xavier

CPF do Cliente (apenas números, sem pontos ou traços):
12345678910
```

Imagem 5. Print do cadastro de clientes, com um exemplo de preenchimento.

No momento atual, não há verificação de CPF duplicado ou inválido e não é possível criar um cliente com fidelidade — dadas as restrições de tempo e complexidade do projeto —, sendo essas implementações futuras para o sistema do Tia Rosa Coffee Shop.

Com as informações coletadas, o cliente é cadastrado no sistema e o usuário é apresentado a uma tela de feedback.

```
Prompt de Comando - pythor x + v
*****
                          Cadastro de Cliente:
                          -----

Preencha com as informações do Cliente:
Nome do Cliente:
Lais Sales Xavier

CPF do Cliente (apenas números, sem pontos ou traços):
12345678910

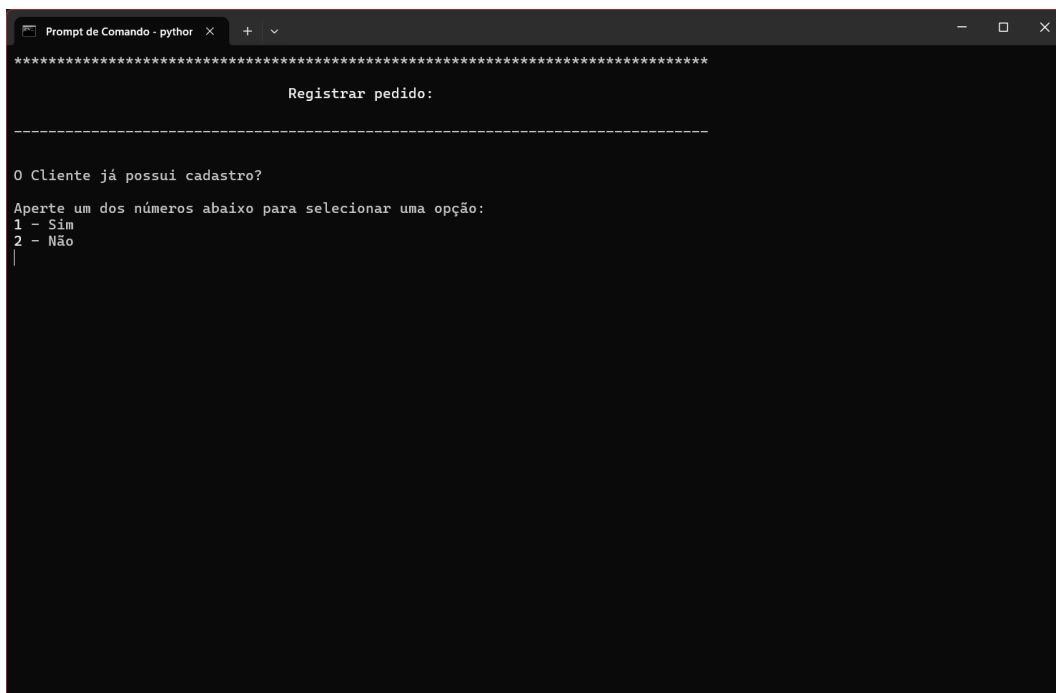
Cliente "Lais Sales Xavier" cadastrado(a) com sucesso!
*****
Pressione 0 para sair.
|
```

Imagem 6. Print do feedback de cadastro do cliente, que se refere ao texto logo abaixo das informações de cadastro e à opção de Sair.

5.4. Pedido (pedido.py)

Fazer um pedido é a funcionalidade mais extensa do programa. Ela contém vários caminhos a depender do cadastro do cliente, cada um seguindo sua lógica e contendo suas opções.

A primeira etapa é a definição do cliente que está fazendo esse pedido. Ele é cadastrado ou não? O cliente não é obrigado a se identificar para consumir, podendo seguir direto para o pedido.



```
Prompt de Comando - pythor x + v
*****
Registrar pedido:
-----

0 Cliente já possui cadastro?
Aperte um dos números abaixo para selecionar uma opção:
1 - Sim
2 - Não
|
```

Imagem 7. Print da definição do cadastro do cliente que realizará o pedido.

1. Caso o cliente seja cadastrado, seguirá para a tela de identificação do cliente, onde é pedido seu CPF:

```
Prompt de Comando - pythor x + v
*****
Registrar pedido:
-----
Informe o CPF do cliente (apenas números, sem pontos ou traços):
12345678910|
```

Imagem 8. Print da coleta do CPF do cliente cadastrado, para seguir com o pedido.

- a. Caso informe um CPF inexistente no sistema, será apresentado a uma tela com o feedback de que o cliente não foi encontrado:

```
Prompt de Comando - pythor x + v
*****
Registrar pedido:
-----
Cliente não encontrado
Pressione 0 para voltar.
|
```

Imagem 9. Print da tela que informa que o cliente não foi encontrado. Essa tela oferece uma opção para voltar para o início do fluxo de pedido.

2. Caso não seja cadastrado, será apresentada uma tela com a opção de cadastrar ou não esse cliente:

```
Prompt de Comando - pythor x + v
*****
Registrar pedido:
-----

Deseja cadastrar o Cliente?
Aperte um dos números abaixo para selecionar uma opção:
1 - Sim
2 - Não
|
```

Imagem 10. Print da opção de cadastrar ou não o cliente antes de realizar o pedido. Caso o usuário deseje cadastrar, será redirecionado para o fluxo de cadastro (consulte o índice 5.3.).

Após essa árvore de decisões iniciais — podendo o cliente ter sido identificado pelo CPF, cadastrado naquele momento, ou não desejado se identificar — o fluxo segue para o registro dos produtos no pedido.

```
Prompt de Comando - pythor x + v
*****
Registrar pedido:
-----

Informe, um por vez, o código (apenas o número) dos produtos a serem adicionados no pedido.
Para finalizar, digite 0.
-----

Código do produto:
1
Código do produto:
4
Código do produto:
4
Código do produto:
8
Código do produto:
2
Código do produto:
6
Código do produto:
9
Digite um código válido. Nenhum produto foi adicionado ao pedido.
Código do produto:
|
```

Imagem 11. Print do registro do pedido, com um exemplo de preenchimento. No cardápio existem os produtos com códigos de 1 a 8, e por isso é apresentado um erro quando o usuário digita o número 9. Para finalizar o registro, basta digitar o número 0.

Ao encerrar o registro, o usuário será apresentado a uma tela de resumo do pedido. É apresentada uma tabela com os produtos, suas quantidades, valores individuais e totais; a tabela é gerada usando o pacote externo “*tabulate*”. Abaixo, o valor total do pedido — já com os cálculos de eventuais descontos por fidelidade — e os pontos acumulados. É apresentada uma opção ao usuário de “Finalizar” ou “Cancelar” o pedido, em caso de desistência do cliente.

```
*****
Resumo do pedido:
-----
+-----+-----+-----+-----+-----+
| Código | Nome           | Quantidade | Valor Unitário | Valor Total |
+-----+-----+-----+-----+-----+
| 1      | Café Expresso  | 1          | R$2.90         | R$2.90      |
+-----+-----+-----+-----+-----+
| 4      | Pão na Chapa   | 2          | R$5.90         | R$11.80     |
+-----+-----+-----+-----+-----+
| 8      | Fatia de Bolo de Chocolate | 1          | R$7.90         | R$7.90      |
+-----+-----+-----+-----+-----+
| 2      | Pingado        | 1          | R$3.30         | R$3.30      |
+-----+-----+-----+-----+-----+
| 6      | Misto Quente   | 1          | R$9.90         | R$9.90      |
+-----+-----+-----+-----+-----+

Valor Total: R$32.22
Pontos acumulados: 60

Deseja finalizar ou cancelar o pedido?

Aperte um dos números abaixo para selecionar uma opção:
1 - Finalizar
2 - Cancelar
|
```

Imagem 12. Print do resumo do pedido. O pedido no print foi registrado para um cliente com cadastro, e por isso ele teve um desconto de 10% no valor total do pedido (de R\$35,80 por R\$32,22) e acumulou 60 pontos (6 produtos no pedido). Ao finalizar ou cancelar o pedido, o usuário retorna para o *Menu Principal*.

1. Caso o cliente decida finalizar o pedido, é somado ao seu cadastro +1 pedido e os pontos acumulados (apenas para clientes identificados);
2. Caso o cliente decida cancelar, o pedido é deletado do sistema.

6. CONCLUSÃO

O projeto atende às necessidades básicas do Tia Rosa Coffee Shop, fornecendo um sistema de gestão interna para os pedidos, um cardápio organizado e estruturado com todas as informações que os clientes precisam para tomar suas decisões, e um sistema de cadastro que permite que os clientes se identifiquem e recebam automaticamente descontos e promoções.

Para o futuro do projeto, a empresa pode investir na expansão dessas funcionalidades. O cardápio hoje é integrado no sistema, e por ser mais utilizado por clientes que por gestão interna, pode-se separá-lo para rodar exclusivamente em *tablets* nas mesas dos clientes. O sistema de cadastro ainda é simples, podendo a empresa expandir essa funcionalidade com mais verificações (CPF duplicado ou inválido), fidelidades e promoções (atualmente apenas dá direito à desconto de 10%), etc. Com o pedido, uma expansão possível seria implementar uma visualização dos produtos para facilitar o registro pelos funcionários.

De forma geral, o aprendizado na criação do projeto foi um desenvolvimento excelente nas minhas habilidades de programação. Pensar em como fazer telas em loop para lidar com erros de input (letras, teclas especiais, etc.) e árvores de opções foi o maior desafio — que inicialmente sofri com laços de repetição mas consegui um sistema consistente com funções recursivas. Aprendi métodos novos no Python procurando formas diferentes de resolver problemas para os meus casos específicos, passei tempo reformulando e reorganizando pedaços de código extensos ou confusos, e de forma geral obtive um resultado que me satisfez bastante.